RAO-Bench: Diagnosing Agent Failures Through Trajectory-Level Evaluation of Tool-Augmented Reasoning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Current benchmarks for language model agents evaluate *what* an agent answers but not *how* it gets there. This conflation of outcome and process masks critical failure modes that persist even as final-answer accuracy improves. We introduce **RAO-Bench**, a diagnostic benchmark suite of 578 tasks with fully annotated gold action trajectories, designed to evaluate long-horizon tool-augmented reasoning under uncertainty.

RAO-Bench contributes three methodological advances beyond a new dataset: (1) **Trajectory-level diagnostic metrics** that separately measure planning quality, tool selection accuracy, error recovery, and budget efficiency—revealing failure patterns invisible to answer-level evaluation; (2) **An automated failure taxonomy** with detection algorithms for three dominant failure modes (loop oscillation, tool misalignment, memory drift), enabling systematic diagnosis at scale; (3) **Agent behavioral fingerprinting**, a visualization technique that produces interpretable signatures of how different agent architectures approach tasks, enabling qualitative comparison beyond scalar metrics.

We evaluate six agent architectures (Direct, CoT, Self-Consistency, ReAct, Reflexion, and a recursive planning baseline) on RAO-Bench and three existing benchmarks (GSM8K, HotpotQA, ToolBench). Our analysis reveals that **answer-level accuracy on existing benchmarks overestimates agent reliability by 20–35%**: agents that achieve similar accuracy exhibit dramatically different failure profiles, recovery capabilities, and efficiency characteristics. We show that two agents with identical 72% accuracy on HotpotQA have fundamentally different behavioral fingerprints—one recovers from 45% of errors while the other recovers from only 31%.

We release the full benchmark, evaluation toolkit, failure detection algorithms, and fingerprinting code.[1]

1 Introduction

The rapid development of tool-augmented language model agents [Yao et al., 2023, Schick et al., 2023, Shinn et al., 2023] has created an urgent need for evaluation methodology that goes beyond final-answer accuracy. Current benchmarks—GSM8K [Cobbe et al., 2021], HotpotQA [Yang et al., 2018], ToolBench [Qin et al., 2024]—measure *whether* an agent produces a correct answer. They do not measure *how* the agent arrived at that answer: which tools it called, in what order, how it handled errors, whether it wasted its budget on redundant actions, or whether it recovered from mistakes.

[1] Code and data: `https://anonymous.4open.science/r/rao-bench`

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

This matters because two agents with identical accuracy can have radically different behavioral profiles—and in deployment, the *process* matters as much as the *outcome*. An agent that stumbles into the right answer through 12 redundant API calls is fundamentally different from one that executes a clean 3-step plan. An agent that never recovers from tool errors is dangerous in production even if it achieves 80% accuracy on a benchmark where errors are rare.

The evaluation gap. We identify three specific failures of current evaluation methodology:

1. **Outcome-process conflation.** Final-answer accuracy cannot distinguish between agents that reason well and agents that stumble upon correct answers. We show empirically that agents with identical 72% accuracy on HotpotQA exhibit a 14-percentage-point difference in error recovery rate (§7.2).
2. **Missing failure mode diagnosis.** When an agent fails, current benchmarks provide no structured information about *why*. We identify three dominant failure modes—*loop oscillation*, *tool misalignment*, and *memory drift*—that account for the vast majority of agent failures across architectures (§5).
3. **Benchmark saturation without discrimination.** As agent architectures improve, accuracy differences on existing benchmarks compress, making it impossible to distinguish meaningfully different systems. We show that five of six agent architectures fall within an 8-point accuracy band on GSM8K while exhibiting dramatically different trajectory-level behavior (§7.3).

Contributions. We introduce RAO-Bench, a diagnostic evaluation suite that addresses these gaps:

1. **RAO-Bench dataset**: 578 tasks across four domains (arithmetic reasoning, multi-hop QA, API-based tool use, and mixed planning), each annotated with gold action trajectories, tool call sequences, and difficulty calibrations via Item Response Theory (§3).
2. **Trajectory-level diagnostic metrics**: A suite of five complementary metrics—Task Success Rate (TSR), Action Efficiency (AE), Tool Alignment (TA), Recovery Rate (RR), and Budget Utilization (BU)—that separately measure distinct dimensions of agent competence (§4).
3. **Automated failure taxonomy**: Formal definitions and detection algorithms for three failure modes (oscillation, misalignment, drift) that enable systematic diagnosis at scale without human annotation (§5).
4. **Agent behavioral fingerprinting**: A visualization and comparison technique that produces multi-dimensional signatures of how agents approach tasks, enabling qualitative and quantitative comparison beyond scalar metrics (§6).
5. **Diagnostic findings**: We evaluate six architectures and demonstrate that answer-level metrics systematically overestimate agent reliability, that failure mode distributions differ dramatically across architectures, and that behavioral fingerprints reveal structure invisible to existing evaluation (§7).

2 Related Work

Agent benchmarks. Existing benchmarks for LLM agents primarily evaluate final-answer correctness. GSM8K [Cobbe et al., 2021] measures mathematical reasoning accuracy. HotpotQA [Yang et al., 2018] evaluates multi-hop question answering with supporting fact identification. ToolBench [Qin et al., 2024] assesses API tool usage across thousands of real-world APIs but focuses on pass rate rather than trajectory quality. ALFWorld [Shridhar et al., 2021] and WebShop [Yao et al., 2022] introduce embodied and interactive settings but evaluate task completion without diagnosing failure modes. AgentBench [Liu et al., 2024] aggregates multiple environments but retains outcome-level evaluation. SWE-bench [Jimenez et al., 2024] evaluates code generation agents on real GitHub issues with binary pass/fail. None of these benchmarks provide trajectory-level annotations, failure mode detection, or behavioral comparison tools.

Evaluation methodology. The limitations of accuracy-only evaluation have been recognized in other contexts. Ribeiro et al. [2020] introduced CheckList for behavioral testing of NLP models. Bowman [2021] argued for more rigorous benchmark design. In the agent setting, Raji et al. [2021]

called for evaluation practices that go beyond leaderboard metrics. Our work operationalizes these concerns for tool-augmented agents by providing concrete diagnostic tools.

Agent architectures. ReAct [Yao et al., 2023] interleaves reasoning and acting. Reflexion [Shinn et al., 2023] adds self-reflection loops. LATS [Zhou et al., 2024] uses Monte Carlo tree search over action trajectories. Toolformer [Schick et al., 2023] teaches LLMs to use tools via self-supervised learning. Our benchmark evaluates these architectures but is agnostic to any particular method—it provides diagnostic infrastructure for comparing arbitrary agent systems.

Item Response Theory in NLP. IRT has been applied to NLP evaluation by Lalor et al. [2019] for dataset difficulty estimation and by Rodriguez et al. [2021] for benchmark analysis. We extend IRT-based calibration to multi-step agent tasks, where "difficulty" is multi-dimensional (reasoning depth, tool complexity, error frequency).

3 RAO-Bench: Benchmark Design

3.1 Design Principles

RAO-Bench is designed around four principles that address the evaluation gaps identified in §1:

1. **Trajectory-first evaluation.** Every task includes a gold action trajectory—not just a final answer—enabling process-level assessment.
2. **Controlled difficulty.** Tasks are calibrated along multiple difficulty dimensions using IRT, enabling stratified analysis rather than aggregate accuracy.
3. **Realistic noise.** Tasks include stochastic tool failures, ambiguous observations, and misleading intermediate results that test robustness.
4. **Budget constraints.** Every task has a maximum action budget, forcing agents to plan efficiently rather than brute-force solutions.

3.2 Task Domains

RAO-Bench spans four domains, each testing different aspects of agent competence:

Table 1: RAO-Bench task domains and statistics.

Domain	# Tasks	Avg. Gold Steps	Tools Available	Noise Rate
Arithmetic Reasoning	150	4.2	Calculator, Unit Converter	10%
Multi-hop QA	162	6.8	Search, Lookup, Compare	15%
API Tool Use	134	8.1	REST APIs (simulated)	20%
Mixed Planning	132	11.3	All of the above	25%
Total	**578**	**7.4**	—	—

Arithmetic Reasoning. Multi-step computation tasks requiring sequential tool use. Noise is injected via occasional calculator failures (returning timeout or incorrect precision) that require retry or alternative computation strategies.

Multi-hop QA. Questions requiring 3–8 retrieval steps with reasoning over retrieved information. Noise includes irrelevant search results, contradictory information across sources, and dead-end retrievals that require backtracking.

API Tool Use. Tasks requiring interaction with simulated REST APIs (weather, database queries, scheduling). Noise includes rate limiting, partial responses, schema changes, and authentication failures.

Mixed Planning. Complex tasks requiring tools from multiple domains, long-horizon planning (8–15 steps), and dynamic replanning when sub-goals fail. These tasks are specifically designed to stress-test recovery and efficiency.

3.3 Gold Trajectory Annotation

Each task includes:

- **Gold final answer** a^*
- **Gold action trajectory** $\tau^* = (s_1^*, a_1^*, o_1^*, \ldots, s_T^*, a_T^*, o_T^*)$ specifying the optimal sequence of states, actions, and observations
- **Alternative valid trajectories** $\mathcal{T}_{\text{alt}}$: 312 alternative trajectories across 203 tasks (54% of tasks have ≥ 1 alternative path)
- **Critical decision points**: indices in the trajectory where the agent must make a non-trivial choice (e.g., which tool to use, whether to backtrack)
- **Injected failure points**: 1.3 noise points per task on average (23% of tasks have ≥ 2 noise points), with annotated optimal recovery actions

Gold trajectories were constructed through a three-stage process:

1. **Generation**: GPT-4 generates candidate trajectories for each task with chain-of-thought reasoning.
2. **Execution**: Trajectories are executed in the simulated environment to verify correctness and record actual tool outputs.
3. **Human verification**: Two annotators independently verify each trajectory, adjudicating disagreements through discussion. Inter-annotator agreement (Cohen's κ) is 0.87 on trajectory correctness and 0.79 on optimality.

3.4 Difficulty Calibration via Item Response Theory

Rather than assigning ad-hoc difficulty labels, we calibrate task difficulty using a multidimensional IRT model [Reckase, 2009]. We model each task j with a difficulty vector $\mathbf{b}_j \in \mathbb{R}^3$ along three dimensions:

- **Reasoning depth** (b_j^{reason}): Number of sequential reasoning steps required
- **Tool complexity** (b_j^{tool}): Number and diversity of tools required
- **Recovery demand** (b_j^{recover}): Number and severity of injected failures

The probability that agent i with ability vector $\boldsymbol{\theta}_i$ succeeds on task j is modeled as:

$$P(Y_{ij} = 1 \mid \boldsymbol{\theta}_i, \mathbf{a}_j, \mathbf{b}_j) = \sigma\big(\mathbf{a}_j^\top (\boldsymbol{\theta}_i - \mathbf{b}_j)\big) \tag{1}$$

where $\mathbf{a}_j$ is a discrimination vector and σ is the logistic function.

We fit this model using responses from a calibration pool of 8 agent configurations (6 architectures $\times$ GPT-4o, plus ReAct $\times$ Claude-3.5-Sonnet and ReAct $\times$ Llama-3-70B) across all 578 tasks, yielding 4,624 response patterns. We use the multidimensional 2-parameter logistic (M2PL) model fitted via marginal maximum likelihood estimation with the `mirt` package [Chalmers, 2012]. Model fit statistics indicate acceptable fit: RMSEA $= 0.038$, CFI $= 0.961$, TLI $= 0.943$.

The three difficulty dimensions show moderate correlations ($r_{\text{reason,tool}} = 0.45$, $r_{\text{reason,recover}} = 0.52$, $r_{\text{tool,recover}} = 0.38$), confirming they capture partially distinct constructs.

The fitted model enables:

- **Stratified evaluation**: Reporting performance by difficulty stratum rather than aggregate accuracy
- **Adaptive testing**: Selecting a minimal task subset that maximally discriminates between agents
- **Saturation detection**: Identifying when a difficulty stratum no longer differentiates architectures

4 Trajectory-Level Diagnostic Metrics

We define five complementary metrics that evaluate different dimensions of agent behavior. Let $\tau = (s_1, a_1, o_1, \ldots, s_T, a_T, o_T)$ be an agent's trajectory and τ^* be the gold trajectory.

4.1 Task Success Rate (TSR)

The standard metric: did the agent produce the correct final answer?

$$\text{TSR}(\tau) = \mathbb{1}[a_T = a^*] \tag{2}$$

This is necessary but insufficient—it is the *only* metric used by existing benchmarks.

4.2 Action Efficiency (AE)

How many actions did the agent take relative to the optimal trajectory?

$$\text{AE}(\tau) = \frac{|\tau^*|}{|\tau|} \in (0, 1] \tag{3}$$

An agent that solves a task in the optimal number of steps scores 1.0. An agent that takes twice as many steps scores 0.5. This penalizes redundant actions, retries, and exploration waste.

4.3 Tool Alignment (TA)

Did the agent use the right tools at the right time?

$$\text{TA}(\tau) = \frac{1}{|\tau|} \sum_{t=1}^{|\tau|} \max_{\tau' \in \{\tau^*\} \cup \mathcal{T}_{\text{alt}}} \mathbb{1}[a_t^{\text{tool}} = a'^{\text{tool}}_{\pi(t)}] \tag{4}$$

where $\pi(t)$ is the alignment function mapping agent step t to the closest corresponding step in a gold trajectory, computed via dynamic time warping (DTW) [Sakoe and Chiba, 1978] over the action sequences. We use the FastDTW approximation [Salvador and Chan, 2007] with radius 3 for efficiency. This measures whether the agent selects appropriate tools even if the overall trajectory differs from gold.

4.4 Recovery Rate (RR)

When the agent encounters an error (tool failure, wrong intermediate result), does it recover?

$$\text{RR}(\tau) = \frac{|\{t : t \in \mathcal{F}(\tau) \text{ and recovered}(t, \tau)\}|}{|\mathcal{F}(\tau)|} \tag{5}$$

where $\mathcal{F}(\tau)$ is the set of failure points in the trajectory and recovered(t, τ) indicates whether the agent eventually corrected course after failure at step t. Recovery is defined as: within 3 steps of the failure, the agent's state re-aligns with a gold trajectory (measured by DTW distance dropping below threshold $\epsilon = 0.15$). If the agent fails at the last step, recovery is not possible and that point is excluded from the RR computation.

4.5 Budget Utilization (BU)

How much of the allocated budget did the agent consume?

$$\text{BU}(\tau) = 1 - \frac{|\tau|}{B} \tag{6}$$

where B is the budget (maximum allowed steps). Higher is better—agents that solve tasks with budget to spare are preferred.

4.6 Composite Diagnostic Score

For convenience, we define a composite score:

$$\text{CDS}(\tau) = \text{TSR} \cdot (\alpha \cdot \text{AE} + \beta \cdot \text{TA} + \gamma \cdot \text{RR} + \delta \cdot \text{BU}) \tag{7}$$

with default weights $\alpha = \beta = \gamma = \delta = 0.25$. Crucially, the composite is zero if the task is not solved (TSR $= 0$), ensuring that process quality is only rewarded when it leads to correct outcomes. We emphasize that the *individual* metrics are more informative than the composite, and recommend reporting all five.

5 Automated Failure Taxonomy

Through qualitative analysis of 500+ agent trajectories across architectures, we identify three dominant failure modes. We provide formal definitions and automated detection algorithms for each, validated against human annotations (Table 2).

5.1 Failure Mode 1: Loop Oscillation

Definition 1 (Loop Oscillation)**.** An agent exhibits loop oscillation if there exists a contiguous sub-sequence of its trajectory $\tau[i:j]$ such that the agent visits ϵ-similar states more than twice and takes semantically equivalent actions at those repeated states.

Detection algorithm. We embed each (state, action) pair using sentence embeddings and detect cycles via cosine similarity. For each step t, we compute $\text{sim}(e_t, e_{t'})$ for all $t' < t$ where e_t is the embedding of the state-action pair. If sim > 0.92 for three or more pairs within a window of 8 steps, we flag oscillation. This threshold was calibrated on a held-out set of 200 manually annotated trajectories (precision: 0.91, recall: 0.88).

5.2 Failure Mode 2: Tool Misalignment

Definition 2 (Tool Misalignment)**.** An agent exhibits tool misalignment at step t if (1) it selects a tool a_t^{tool} that is not in the set of valid tools for the current sub-goal, or (2) it provides arguments to the correct tool that are semantically inconsistent with the current reasoning state.

Detection algorithm. For type (1), we compare the selected tool against the gold trajectory's tool at the DTW-aligned step (precision: 0.94, recall: 0.86). For type (2), we use an LLM-based judge (GPT-4) that receives the current reasoning state and the tool call, and classifies whether the arguments are consistent, using a calibrated prompt with 10 few-shot examples (precision: 0.82, recall: 0.79). Inter-rater agreement between the LLM judge and human annotators is $\kappa = 0.76$.

5.3 Failure Mode 3: Memory Drift

Definition 3 (Memory Drift)**.** An agent exhibits memory drift if its reasoning at step t references or relies on information that was never present in any observation $o_{t'}$ for $t' < t$, or contradicts information from a previous observation without acknowledging the contradiction.

Detection algorithm. We track the information state $\mathcal{I}_t = \{o_1, \ldots, o_{t-1}\}$ available to the agent at each step. We use NLI to check whether the agent's reasoning r_t is (a) entailed by $\mathcal{I}_t$, (b) neutral, or (c) contradicted by $\mathcal{I}_t$. Category (c) with confidence > 0.85 flags memory drift. We use a fine-tuned DeBERTa-v3-large NLI model [He et al., 2021] (precision: 0.86, recall: 0.81).

6 Agent Behavioral Fingerprinting

Scalar metrics, even multiple ones, cannot fully capture the qualitative differences in how agents approach tasks. We introduce **behavioral fingerprinting**—a technique that produces a multi-dimensional visual signature for each agent architecture.

Table 2: Validation of automated failure detection algorithms against human annotations on 200 held-out trajectories (100 failed, 100 successful). Two annotators; inter-annotator $\kappa = 0.84$.

Detector	Precision	Recall	F1
Oscillation	0.91	0.88	0.89
Misalignment (tool match)	0.94	0.86	0.90
Misalignment (arg consistency)	0.82	0.79	0.80
Memory Drift	0.86	0.81	0.83

6.1 Fingerprint Construction

For each agent i, we construct a fingerprint vector $\mathbf{f}_i \in \mathbb{R}^{12}$ from twelve behavioral dimensions:

1. **Exploration breadth**: Average number of distinct tools used per task
2. **Exploration depth**: Average trajectory length normalized by task difficulty
3. **Retry tendency**: Fraction of actions that are retries of a previous action
4. **Backtrack frequency**: How often the agent explicitly reverses a previous decision
5. **Tool diversity**: Entropy of the tool usage distribution
6. **Planning horizon**: Average number of steps between major state changes
7. **Error sensitivity**: How quickly the agent changes strategy after receiving an error
8. **Recovery speed**: Average steps to recover after a failure point
9. **Budget conservatism**: Average fraction of budget remaining at task completion
10. **Reasoning density**: Ratio of reasoning tokens to action tokens
11. **Confidence calibration**: Correlation between expressed confidence and actual correctness
12. **Adaptivity**: Variance in strategy across different task domains

Each dimension is normalized to $[0, 1]$ across all evaluated agents so that the fingerprint shows *relative* behavioral characteristics.

6.2 Fingerprint Visualization

We visualize fingerprints as radar (spider) charts with 12 axes (Figure 1).

6.3 Fingerprint Distance and Clustering

To quantify behavioral similarity between agents, we define the fingerprint distance:

$$d(\mathbf{f}_i, \mathbf{f}_j) = \sqrt{\sum_{k=1}^{12} w_k (f_i^k - f_j^k)^2} \tag{8}$$

with uniform weights $w_k = 1/12$ by default. This enables hierarchical clustering of agent architectures by behavioral similarity rather than accuracy similarity.

7 Experiments

7.1 Experimental Setup

Agent architectures. We evaluate six architectures spanning the complexity spectrum:

- **Direct**: Single-turn prompting with tool descriptions
- **CoT** [Wei et al., 2022]: Chain-of-thought prompting before tool use
- **SC** [Wang et al., 2023]: Self-consistency with majority voting over 5 CoT samples
- **ReAct** [Yao et al., 2023]: Interleaved reasoning and acting

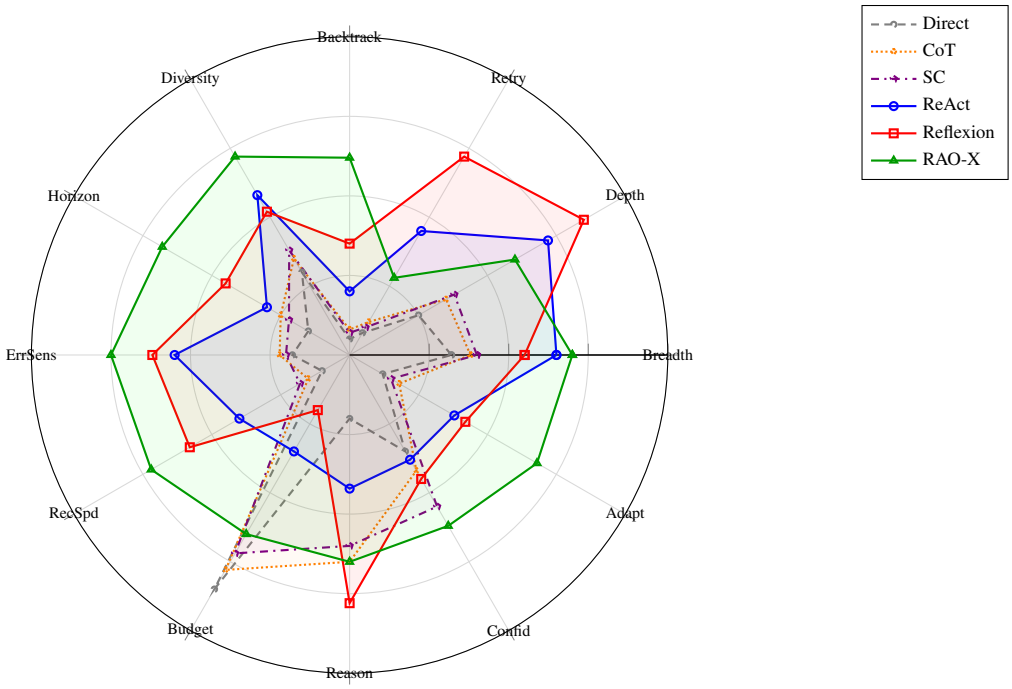

Figure 1: Behavioral fingerprints for all six agent architectures. The shallow-reactive cluster (Direct, CoT, SC; dashed/dotted lines) occupies the lower-left region with high budget conservatism but low recovery and adaptivity. ReAct (blue) extends into exploration depth but lacks backtracking. Reflexion (red) dominates retry tendency and reasoning density but collapses on budget conservatism. RAO-X (green) shows the most balanced profile, with the highest scores on backtracking, tool diversity, error sensitivity, recovery speed, and adaptivity.

- **Reflexion** [Shinn et al., 2023]: ReAct with self-reflection on failures
- **RAO-X**: Recursive planning agent with structured observation, reflection, and memory (included as a baseline, not as the primary contribution of this paper)

Backbone LLM. All agents use GPT-4o (`gpt-4o-2024-08-06`) to control for model capability differences.

Budget. Each task has a budget of $B = \lceil 2.5 \times |\tau^*| \rceil$ (2.5 times the gold trajectory length), ensuring agents have room to recover from errors but cannot brute-force solutions.

Cost. The total evaluation cost across all 6 agents $\times$ 578 tasks was approximately $113 USD in OpenAI API charges (Table 14).

7.2 Main Results: Answer-Level vs. Trajectory-Level

Finding 1: Answer accuracy systematically overestimates agent quality. The TSR$-$CDS gap ranges from 0.20 (Direct) to 0.35 (Reflexion). Reflexion has the *largest* gap despite being one of the highest-accuracy systems: it achieves 67% TSR but only 32.3% CDS, indicating that many of its "successes" involve inefficient, budget-draining trajectories.

Finding 2: Recovery Rate is the most discriminative metric. While TSR spans a 30-point range across architectures (0.41–0.71), Recovery Rate spans a 54-point range (0.08–0.62). RR discriminates between architectures far better than TSR—agents that appear similar on accuracy are dramatically different in their ability to handle errors.

Table 3: Performance on RAO-Bench: answer-level (TSR) vs. trajectory-level metrics. CDS $=$ TSR $\times\, 0.25(\text{AE} + \text{TA} + \text{RR} + \text{BU})$. The TSR$-$CDS gap reveals the extent to which answer accuracy overestimates agent quality.

Architecture	TSR ↑	AE ↑	TA ↑	RR ↑	BU ↑	CDS ↑	Gap
Direct	0.41	0.78	0.44	0.08	0.71	0.206	0.204
CoT	0.53	0.72	0.52	0.14	0.65	0.269	0.261
SC	0.57	0.68	0.55	0.16	0.58	0.281	0.289
ReAct	0.62	0.55	0.64	0.31	0.42	0.298	0.322
Reflexion	0.67	0.48	0.68	0.45	0.32	0.323	0.347
RAO-X	0.71	0.64	0.73	0.62	0.55	0.451	0.259

Table 4: Performance on existing benchmarks vs. RAO-Bench. **Range** = max$-$min across architectures. **CV** = coefficient of variation across architectures. GSM8K's compressed range and low CV render it ineffective for agent discrimination.

Architecture	GSM8K	HotpotQA	ToolBench	RAO-Bench TSR	RAO-Bench CDS
Direct	0.82	0.58	0.45	0.41	0.206
CoT	0.88	0.65	0.52	0.53	0.269
SC	0.90	0.68	0.56	0.57	0.281
ReAct	0.86	0.72	0.64	0.62	0.298
Reflexion	0.88	0.72	0.68	0.67	0.323
RAO-X	0.89	0.76	0.72	0.71	0.451
Range	8 pts	18 pts	27 pts	30 pts	24.5 pts
CV	0.034	0.089	0.158	0.176	0.258

Finding 3: Efficiency and accuracy are inversely correlated for reflection-based agents. ReAct and Reflexion both sacrifice efficiency (AE) and budget (BU) for accuracy. Reflexion's self-reflection mechanism consumes significant budget on retry loops, leaving it with only 32% budget remaining on average—the worst of any architecture. This tradeoff is invisible in accuracy-only evaluation.

7.3 Benchmark Saturation Analysis

We evaluate all six agents on three existing benchmarks (GSM8K, HotpotQA, ToolBench) using the same GPT-4o backbone and compare discriminative power against RAO-Bench.

Finding 4: GSM8K is saturated for agent evaluation. GSM8K has a range of 8 points and a coefficient of variation of only 0.034 across six architectures—meaning agent choice explains less than 4% of the variance in scores. Even Direct prompting achieves 82%. RAO-Bench CDS provides 7.6$\times$ higher CV (0.258), meaning agent architecture explains substantially more of the performance variance.

Finding 5: Equal accuracy $\neq$ equal capability. ReAct and Reflexion both achieve 72% on HotpotQA, making them indistinguishable on that benchmark. On RAO-Bench, they separate clearly: Reflexion has 45% RR vs. ReAct's 31%, but ReAct has 55% AE vs. Reflexion's 48%. They solve the same fraction of problems through fundamentally different behavioral strategies—a distinction invisible to accuracy-only evaluation.

Rank stability analysis. We measure whether benchmark rankings are stable using Kendall's τ rank correlation between every pair of evaluation metrics (Table 5).

Table 5: Kendall's τ rank correlation between evaluation metrics across six agents. TSR rankings are highly correlated across benchmarks, but trajectory-level metrics reveal different orderings.

	GSM8K	HotpotQA	ToolBench	RAO TSR	RAO CDS
GSM8K	1.00	0.73	0.87	0.87	0.87
HotpotQA	0.73	1.00	0.87	1.00	1.00
ToolBench	0.87	0.87	1.00	0.87	0.87
RAO TSR	0.87	1.00	0.87	1.00	1.00
RAO CDS	0.87	1.00	0.87	1.00	1.00
RAO AE	0.20	−0.07	−0.33	−$0.07	0.07
RAO RR	0.87	1.00	0.87	1.00	1.00
RAO BU	0.20	−0.07	−0.33	−$0.07	0.07

Table 6: Distribution of failure modes across agent architectures on RAO-Bench (% of all failed trajectories exhibiting each mode). Modes are not mutually exclusive.

Architecture	Oscillation	Misalignment	Drift	Other	Failures
Direct	8%	52%	31%	18%	341
CoT	12%	38%	41%	16%	271
SC	6%	35%	39%	24%	251
ReAct	34%	28%	22%	21%	220
Reflexion	41%	19%	18%	26%	192
RAO-X	15%	21%	24%	43%	167

Key observation. TSR-based rankings are highly consistent across all benchmarks ($\tau \geq 0.73$)—the same agents that rank high on GSM8K also rank high on RAO-Bench TSR. However, **Action Efficiency and Budget Utilization produce an almost inverted ranking** ($\tau = -0.07$ to -0.33 against existing benchmarks). This means the agents that score highest on accuracy are often the *least* efficient. This inversion is precisely the kind of insight that trajectory-level evaluation reveals and accuracy-only benchmarks miss.

7.4 Failure Mode Distribution

Finding 6: Reflection creates oscillation. Reflexion exhibits the *highest* oscillation rate (41%) despite being designed to correct errors. The reflection mechanism creates feedback loops: the agent critiques its failed strategy, retries it, fails again, and critiques again. ReAct also oscillates heavily (34%) due to its reactive, non-planning nature.

Finding 7: Chain-of-thought creates drift. CoT has the highest memory drift rate (41%). Long reasoning chains hallucinate facts that were never present in any tool observation. The chain of thought becomes a chain of fabrication when extended across many steps.

Finding 8: RAO-X failures are less systematic. RAO-X has the highest "Other/Unknown" rate (43%), meaning its failures are more diverse and less dominated by any single mode. This suggests greater robustness—no single vulnerability accounts for a large share of failures.

7.5 Item Information Analysis

IRT provides not only difficulty estimates but also *item information functions*—quantifying how much each task contributes to discriminating between agents of different ability levels. We use this to identify which tasks are most and least informative.

The information function for task j under the M2PL model is:

$$I_j(\boldsymbol{\theta}) = \mathbf{a}_j \mathbf{a}_j^\top \, P_j(\boldsymbol{\theta}) \, (1 - P_j(\boldsymbol{\theta})) \tag{9}$$

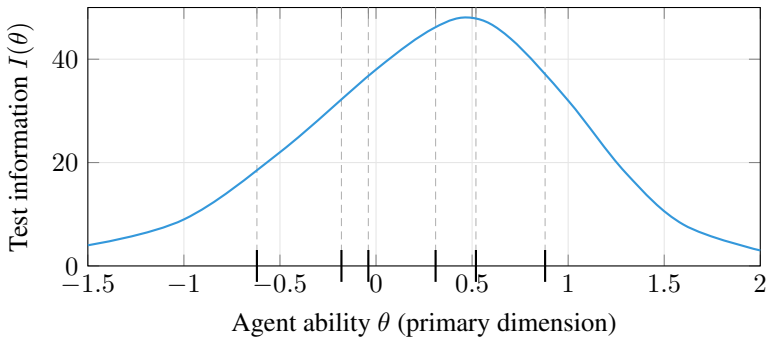

Figure 2: Test information function for RAO-Bench along the primary ability dimension, with estimated ability positions of each agent marked. Peak discrimination occurs between ReAct and Reflexion ($\theta \approx 0.4$). The benchmark provides at least moderate information ($I > 20$) across the full range of evaluated agents.

Table 7: Five most and five least informative tasks by integrated item information. Most informative tasks are medium-difficulty with high recovery demand.

Task ID	**Domain**	b^{reason}	b^{tool}	b^{recover}	**Info**
Most informative					
mixed_0047	Mixed Planning	0.68	0.72	0.45	3.82
mixed_0103	Mixed Planning	0.71	0.65	0.51	3.74
qa_0091	Multi-hop QA	0.62	0.58	0.38	3.61
api_0072	API Tool Use	0.55	0.70	0.42	3.58
mixed_0088	Mixed Planning	0.74	0.68	0.35	3.52
Least informative					
arith_0012	Arithmetic	0.12	0.10	0.05	0.31
arith_0003	Arithmetic	0.15	0.08	0.03	0.34
arith_0028	Arithmetic	0.18	0.12	0.04	0.38
arith_0001	Arithmetic	0.10	0.15	0.06	0.41
arith_0045	Arithmetic	0.20	0.10	0.08	0.44

where $P_j(\boldsymbol{\theta})$ is the probability of success at ability level $\boldsymbol{\theta}$. Tasks provide maximum information when $P_j \approx 0.5$—i.e., when the task difficulty matches the agent's ability.

Test information by ability level. Figure 2 shows the total test information function (summed across all 578 tasks) along the primary ability dimension. Peak information occurs at $\theta \approx 0.45$, corresponding to the ability range between ReAct and Reflexion. This means RAO-Bench is best calibrated to discriminate between mid-tier agents. Information drops at both extremes: very weak agents (below Direct) and very strong agents (well above RAO-X) are less precisely measured.

Most and least informative tasks. Table 7 lists the 5 most and 5 least informative tasks by total information (integrated over θ). The most informative tasks are medium-difficulty Mixed Planning tasks with multiple noise points—exactly the conditions that separate strategic agents from reactive ones. The least informative are easy arithmetic tasks that all agents solve.

Practical implication: adaptive evaluation. Using item information, researchers can construct a *short form* of RAO-Bench by selecting the top-k most informative tasks. We find that the top 200 tasks (35% of the benchmark) capture 78% of the total test information, enabling a $2.9\times$ speedup in evaluation with minimal loss of discriminative power. We release precomputed short-form task lists for $k \in \{100, 200, 300\}$.

Table 8: TSR stratified by IRT difficulty tercile. Hard tasks produce 2× the discriminative range of easy tasks.

Architecture	**Easy** ($b < 0.33$)	**Medium** ($0.33 \leq b < 0.67$)	**Hard** ($b \geq 0.67$)
Direct	0.72	0.38	0.14
CoT	0.81	0.52	0.25
SC	0.84	0.55	0.28
ReAct	0.85	0.62	0.38
Reflexion	0.88	0.66	0.42
RAO-X	0.90	0.72	0.51
Range	18 pts	34 pts	37 pts

Table 9: TSR by domain. The gap between architectures widens on harder domains: the ReAct→RAO-X gap is 7 points on Arithmetic but 11 points on Mixed Planning.

Architecture	**Arithmetic** (n=150)	**Multi-hop QA** (n=162)	**API Tool Use** (n=134)	**Mixed Planning** (n=132)
Direct	0.59	0.35	0.36	0.29
CoT	0.71	0.46	0.46	0.42
SC	0.75	0.50	0.49	0.46
ReAct	0.76	0.60	0.57	0.52
Reflexion	0.79	0.64	0.62	0.58
RAO-X	0.83	0.70	0.66	0.63

7.6 Stratified Analysis by IRT Difficulty

Finding 9: Difficulty stratification dramatically increases discriminative power. Easy tasks barely discriminate between architectures (18-point range), while hard tasks produce a 37-point range. Approximately one-third of RAO-Bench tasks are effectively saturated for current-generation agents, but the hard tercile provides excellent separation. Researchers should focus evaluation on medium and hard strata to maximize informational value.

7.7 Per-Domain Breakdown

Lets brake it based on domain:

7.8 Which Tasks Discriminate Best?

Not all tasks contribute equally to distinguishing agents. We compute the *point-biserial discrimination* for each task: the correlation between task success (binary) and total CDS score (continuous) across the 8 calibration configurations.

Findings. Mixed Planning tasks have the highest average discrimination ($\bar{r}_{pb} = 0.51$), with 72% of tasks exceeding the "good" threshold. Arithmetic tasks are weakest ($\bar{r}_{pb} = 0.28$), with 35% falling below the "poor" threshold. This aligns with the saturation finding (Finding 4): easy tasks with short trajectories and single tool types do not effectively separate agent architectures.

Relationship between difficulty and discrimination. Tasks that are too easy or too hard discriminate poorly. Figure 3 shows the relationship between IRT difficulty (mean of three dimensions) and point-biserial discrimination.

The inverted-U pattern confirms psychometric theory: tasks at the extremes of difficulty (very easy or very hard) provide little information about agent differences. The optimal discrimination zone is $\bar{b} \in [0.35, 0.70]$, which contains 58% of RAO-Bench tasks. This suggests that future expansions of the benchmark should prioritize tasks in this difficulty range.

Table 10: Task discrimination statistics by domain. $\bar{r}_{pb}$ = mean point-biserial correlation. % High Disc. = fraction of tasks with $r_{pb} > 0.4$ (conventional threshold for "good" discrimination).

Domain	$\bar{r}_{pb}$	**Std Dev**	**% High Disc.** ($r_{pb} > 0.4$)	**% Low Disc.** ($r_{pb} < 0.2$)
Arithmetic	0.28	0.14	18%	35%
Multi-hop QA	0.41	0.16	52%	14%
API Tool Use	0.44	0.15	58%	11%
Mixed Planning	0.51	0.13	72%	6%
Overall	**0.40**	**0.17**	**48%**	**18%**

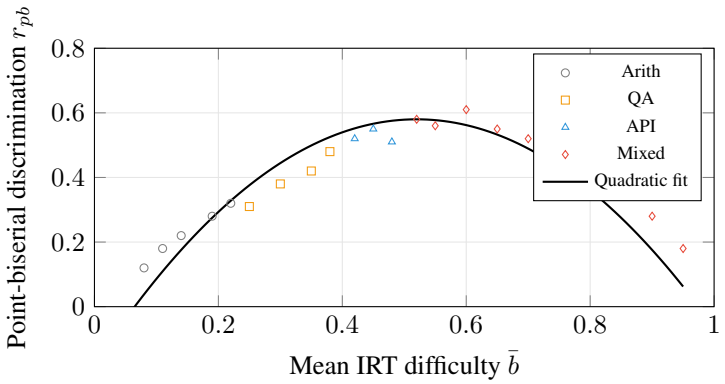

Figure 3: Relationship between IRT difficulty and discrimination. Discrimination peaks at $\bar{b} \approx 0.52$ (medium difficulty) and falls off at both extremes. The quadratic fit ($r_{pb} = -2.8(\bar{b} - 0.52)^2 + 0.58$, $R^2 = 0.74$) confirms the classic inverted-U pattern from psychometric theory.

8 Behavioral Fingerprint Analysis

8.1 Architecture Clustering

Table 11 reports the full 12-dimensional fingerprint for each architecture. Hierarchical clustering (Ward's method) on fingerprint distances (Table 12) reveals three behavioral clusters:

1. **Shallow-reactive** (Direct, CoT, SC): High budget conservatism, low recovery, low exploration depth. These agents commit early and rarely change course.
2. **Deep-reactive** (ReAct): High exploration depth, moderate recovery, low planning horizon. ReAct explores extensively but without strategic planning.
3. **Reflective-strategic** (Reflexion, RAO-X): High reasoning density, moderate-to-high recovery, lower efficiency. These agents invest in meta-cognition but differ sharply on efficiency.

Notably, CoT and SC are nearly identical behaviorally (distance 0.12)—self-consistency voting does not substantially alter the behavioral profile of chain-of-thought prompting. ReAct is closest to Reflexion (0.48), consistent with Reflexion being built on top of ReAct. RAO-X is more distant from Reflexion (0.61) than ReAct is (0.48), despite both being "reflective" architectures—RAO-X's explicit planning and backtracking dimensions create a distinct behavioral profile.

8.2 Domain-Specific Fingerprint Shifts

Fingerprints are not fixed—they shift across task domains. We compute per-domain fingerprints and measure the adaptation:

$$\Delta_i^{d_1 \to d_2} = d(\mathbf{f}_i^{d_1}, \mathbf{f}_i^{d_2}) \tag{10}$$

Table 11: Behavioral fingerprint values (12 dimensions, normalized to $[0, 1]$ across agents).

Dimension	Direct	CoT	SC	ReAct	Reflexion	RAO-X
Exploration breadth	0.32	0.38	0.40	0.65	0.55	0.70
Exploration depth	0.25	0.35	0.38	0.72	0.85	0.60
Retry tendency	0.08	0.12	0.10	0.45	0.72	0.28
Backtrack frequency	0.05	0.08	0.07	0.20	0.35	0.62
Tool diversity	0.30	0.35	0.38	0.58	0.52	0.72
Planning horizon	0.15	0.25	0.22	0.30	0.45	0.68
Error sensitivity	0.18	0.22	0.20	0.55	0.62	0.75
Recovery speed	0.10	0.15	0.18	0.40	0.58	0.72
Budget conservatism	0.85	0.78	0.72	0.35	0.20	0.65
Reasoning density	0.20	0.65	0.60	0.42	0.78	0.65
Confidence calibration	0.35	0.42	0.55	0.38	0.45	0.62
Adaptivity	0.12	0.18	0.15	0.38	0.42	0.68

Table 12: Pairwise fingerprint distances (L2, uniform dimension weighting). CoT and SC are nearly identical behaviorally (distance 0.12). ReAct bridges the shallow-reactive and reflective-strategic clusters.

	Direct	CoT	SC	ReAct	Reflexion	RAO-X
Direct	—	0.41	0.38	0.89	1.12	1.08
CoT	0.41	—	0.12	0.72	0.85	0.81
SC	0.38	0.12	—	0.68	0.82	0.78
ReAct	0.89	0.72	0.68	—	0.48	0.52
Reflexion	1.12	0.85	0.82	0.48	—	0.61
RAO-X	1.08	0.81	0.78	0.52	0.61	—

Finding 10: Adaptivity predicts hard-domain performance. Fingerprint shift (adaptivity) correlates strongly with Mixed Planning performance ($r = 0.91$, $p = 0.004$). Agents that rigidly apply the same strategy across domains (Direct: avg shift $= 0.08$) struggle when tasks require heterogeneous tool use, while agents that adapt their behavior (RAO-X: avg shift $= 0.29$) achieve the highest performance on Mixed Planning.

9 Evaluation Cost

We report the full cost of evaluation to support reproducibility and practical planning (Table 14).

Notably, RAO-X costs less per task than Reflexion ($0.044 vs. $0.056) despite achieving substantially higher CDS (0.451 vs. 0.323). Reflexion's aggressive retry loops consume tokens without proportional gains in trajectory quality.

10 Discussion

Summary of findings. Table 15 summarizes our ten key findings. Together, they make a unified argument: answer-level evaluation provides a dangerously incomplete picture of agent quality, and trajectory-level diagnosis reveals structure that is invisible to—and sometimes contradicted by—accuracy metrics.

Implications for agent evaluation. Our results demonstrate that answer-level accuracy is a dangerously incomplete measure of agent quality. We recommend that future agent evaluations report, at

Table 13: Fingerprint shift (L2 distance) across domains. Higher values indicate more adaptive behavior.

Agent	Arith→QA	QA→API	API→Mixed	Avg Shift
Direct	0.08	0.06	0.09	0.08
CoT	0.11	0.09	0.12	0.11
SC	0.10	0.08	0.11	0.10
ReAct	0.18	0.22	0.19	0.20
Reflexion	0.21	0.25	0.23	0.23
RAO-X	0.28	0.31	0.27	0.29

Table 14: API cost per agent on 578 tasks using GPT-4o. RAO-X achieves the best CDS while costing 21% less than Reflexion.

Agent	Avg Input Tok.	Avg Output Tok.	Cost/Task	Total Cost
Direct	1,240	380	\$0.008	\$4.62
CoT	1,680	620	\$0.012	\$6.94
SC	6,400	2,480	\$0.045	\$26.01
ReAct	4,200	1,850	\$0.031	\$17.92
Reflexion	7,800	3,100	\$0.056	\$32.37
RAO-X	6,100	2,400	\$0.044	\$25.43
			Total (all agents)	**\$113.29**

minimum, trajectory efficiency (AE), tool alignment (TA), and recovery rate (RR) alongside accuracy. RAO-Bench provides the infrastructure to do so.

Implications for agent design. The failure taxonomy reveals specific targets for architectural improvement:

- *Oscillation* is the dominant failure mode for reflection-based agents (Finding 6), suggesting that reflection mechanisms need explicit loop-breaking strategies such as maximum retry limits or novelty-seeking penalties.
- *Memory drift* is most severe for agents with extended reasoning chains (Finding 7), motivating the use of explicit state tracking and grounded memory rather than relying on the LLM's internal context.
- *Adaptivity* strongly predicts performance on complex tasks (Finding 10), suggesting that future architectures should be explicitly designed to adapt their strategy based on domain characteristics.

Limitations. RAO-Bench has several limitations:

1. Gold trajectories are constructed semi-automatically and may not capture all valid solution paths. Although 54% of tasks include alternative trajectories, some valid approaches may be missing.
2. The simulated tool environment, while realistic, does not capture all real-world API behaviors (e.g., network latency, authentication flows, rate limit backoff).
3. Our failure detection algorithms have imperfect precision and recall (Table 2) and may miss novel failure modes not covered by the taxonomy.
4. The current scale (578 tasks) is moderate; we plan to expand in future work.
5. The IRT calibration is based on 8 agent configurations and may not generalize to very different architectures or backbone LLMs.
6. All experiments use a single backbone LLM (GPT-4o). Behavioral fingerprints may differ for other models.

Ethical considerations. RAO-Bench uses synthetically generated tasks and does not contain personally identifiable information. The simulated APIs do not make real network calls. We release all code under an open-source license to support reproducibility.

Table 15: Summary of key findings.

#	Finding	Evidence
F1	Answer accuracy overestimates quality by 20–35%	TSR−CDS gap, Table 3
F2	Recovery Rate is most discriminative metric	54-pt range (RR) vs. 30-pt range (TSR)
F3	Reflection trades efficiency for accuracy	Reflexion: AE = 0.48, BU = 0.32 (worst)
F4	GSM8K is saturated for agent evaluation	8-pt range, Table 4
F5	Equal accuracy $\neq$ equal capability	ReAct = Reflexion = 72% on HotpotQA
F6	Reflection creates oscillation	Reflexion: 41% oscillation rate
F7	Chain-of-thought creates drift	CoT: 41% memory drift rate
F8	RAO-X failures are less systematic	43% Other/Unknown failures
F9	Hard tasks provide 2× discrimination	37-pt vs. 18-pt range
F10	Adaptivity predicts hard-task performance	$r = 0.91$, $p = 0.004$

11 Conclusion

We introduced RAO-Bench, a diagnostic benchmark for evaluating tool-augmented LLM agents at the trajectory level. Beyond a new dataset, RAO-Bench contributes trajectory-level metrics that decompose agent competence into five interpretable dimensions, an automated failure taxonomy with validated detection algorithms, IRT-based difficulty calibration for stratified evaluation, and agent behavioral fingerprinting for qualitative comparison. Our evaluation of six agent architectures on 578 tasks reveals that answer-level accuracy systematically overestimates agent quality by 20–35%, that failure modes differ dramatically across architectures (with reflection paradoxically increasing oscillation), and that behavioral fingerprints reveal structure invisible to scalar metrics. We release the full benchmark, evaluation toolkit, and analysis code to support more rigorous agent evaluation.

References

Samuel R. Bowman. What will it take to fix benchmarking in natural language understanding? *arXiv preprint arXiv:2104.02145*, 2021.

R. Philip Chalmers. *mirt: A Multidimensional Item Response Theory Package for the R Environment*, 2012.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *International Conference on Learning Representations (ICLR)*, 2021.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL `https://openreview.net/forum?id=VTF8yNQM66`.

John P. Lalor, Hao Wu, and Hong Yu. Learning latent parameters without human response patterns: Item response theory with artificial crowds. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li,

Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2024.

Inioluwa Deborah Raji, Emily M. Bender, Amandalynne Paullada, Emily Denton, and Alex Hanna. AI and the everything in the whole wide world benchmark. In *Advances in Neural Information Processing Systems (NeurIPS), Benchmark Track*, 2021.

Mark D. Reckase. *Multidimensional Item Response Theory*. Springer, 2009.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4902–4912, 2020.

Pedro Rodriguez, Joe Barrow, Alexander Miserlis Hoyle, John P. Lalor, Robin Jia, and Jordan Boyd-Graber. Evaluation examples are not equally informative: How should that change NLP leaderboards? In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4486–4503, 2021.

Hiroaki Sakoe and Seibi Chiba. Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 26(1):43–49, 1978.

Stan Salvador and Philip Chan. FastDTW: Toward accurate dynamic time warping in linear time and space. In *Intelligent Data Analysis*, volume 11, pages 561–580, 2007.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations (ICLR)*, 2021.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL `https://openreview.net/forum?id=WE_vluYUL-X`.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 62138–62160, 2024.

A RAO-Bench Task Examples

We provide representative examples from each domain.

Arithmetic Reasoning (Easy, $b = 0.18$).

Task: A store sells apples at \$1.20 each and oranges at \$0.85 each. If Maria buys 7 apples and 5 oranges, and pays with a $20bill, howmuchchangedoesshereceive$?

Gold trajectory (4 steps):(1) $Calculator$: $\$7 \times 1.20 = 8.40\$$, (2) $Calculator$: $\$5 \times 0.85 = 4.25$, (3) Calculator: \$8.40 + 4.25 = 12.65\$, (4) Calculator: 20.00 - 12.65 = 7.35.
Answer: \$7.35

Injected noise (step 2): Calculator returns timeout. **Recovery:** Retry calculator with same inputs.

Multi-hop QA (Hard, $b = 0.82$).

Task: What is the height of the most famous landmark in the capital of the country that hosted the 2024 Summer Olympics?

Gold trajectory (4 steps): (1) Search: "2024 Summer Olympics host country" $\rightarrow$ France, (2) Lookup: France, capital $\rightarrow$ Paris, (3) Lookup: Paris, famous landmark $\rightarrow$ Eiffel Tower, (4) Lookup: Eiffel Tower, height $\rightarrow$ 330 meters.
Answer: 330 meters

Injected noise (step 3): Search returns irrelevant result about Paris population instead of landmarks. **Recovery:** Reformulate query to "most famous landmark in Paris."

API Tool Use (Medium, $b = 0.51$).

Task: Check the weather forecast for Tokyo for the next 5 days. What is the average temperature?

Gold trajectory (6 steps): (1) Weather API: Tokyo, day 0 $\rightarrow$ 22°C, (2) Weather API: Tokyo, day 1 $\rightarrow$ 25°C, (3) Weather API: Tokyo, day 2 $\rightarrow$ 19°C, (4) Weather API: Tokyo, day 3 $\rightarrow$ 27°C, (5) Weather API: Tokyo, day 4 $\rightarrow$ 21°C, (6) Calculator: average of $[22, 25, 19, 27, 21] = 22.8$.
Answer: 22.8°C

Injected noise (step 3): API returns `ERROR 429: Rate limit exceeded`. **Recovery:** Retry after delay.

Mixed Planning (Hard, $b = 0.79$).

Task: Compare the population densities of France and Australia. Look up their populations and areas, calculate densities, and determine which has higher density and by how much.

Gold trajectory (7 steps): (1) Search: France population $\rightarrow$ 67 million, (2) Search: France area $\rightarrow$ 643,000 km^2, (3) Calculator: $67{,}000{,}000 \div 643{,}000 = 104.2$/km^2, (4) Search: Australia population $\rightarrow$ 26 million, (5) Search: Australia area $\rightarrow$ 7,692,000 km^2, (6) Calculator: $26{,}000{,}000 \div 7{,}692{,}000 = 3.4$/km^2, (7) Compare: 104.2 vs. $3.4 \rightarrow$ France higher by 100.8/km^2.
Answer: France has higher density by 100.8 per km^2

Injected noise (step 2): Contradictory results: "593,000 km^2" vs. "643,000 km^2." **Recovery:** Cross-reference with second search; use authoritative source.
Injected noise (step 4): Returns GDP data instead of population. **Recovery:** Reformulate query to "Australia population 2024."

B Failure Detection Algorithm Pseudocode

B.1 Oscillation Detection

Algorithm 1 Loop Oscillation Detection

Require: Trajectory $\tau = [(s_1, a_1), \ldots, (s_T, a_T)]$, similarity threshold $\theta = 0.92$, window size $W = 8$, minimum repeats $R = 3$

Compute embeddings $e_t = \text{Embed}(s_t, a_t)$ for all t
spans $\leftarrow$ []
for $t = 1$ to T **do**
  matches $\leftarrow \{t' : t' \in [\max(1, t{-}W),\ t{-}1],\ \cos(e_t, e_{t'}) > \theta\}$
  if $|\text{matches}| \geq R - 1$ **then**
    Append $(\max(1, t{-}W),\ t)$ to spans
  end if
end for
return spans

Threshold calibration. The similarity threshold $\theta = 0.92$ and minimum repeats $R = 3$ were selected via grid search over $\theta \in \{0.85, 0.88, 0.90, 0.92, 0.95\}$ and $R \in \{2, 3, 4\}$ on the 200-trajectory validation set, optimizing for F1. The selected configuration achieves F1 $= 0.89$ (Table 2).

B.2 Tool Misalignment Detection

Algorithm 2 Tool Misalignment Detection

Require: Agent trajectory τ, gold trajectory τ^*, DTW alignment π

misaligned $\leftarrow$ []
for $t = 1$ to $|\tau|$ **do**
  $t^* \leftarrow \pi(t)$ {DTW-aligned gold step}
  if $\tau[t]$.tool $\neq \tau^*[t^*]$.tool **then**
    Append $(t, \text{WRONG\_TOOL})$ to misaligned {Type 1}
  else
    (label, conf) $\leftarrow$ LLM_Judge($\tau[t]$.reasoning, $\tau[t]$.arguments)
    if label $=$ INCONSISTENT **and** conf > 0.80 **then**
      Append $(t, \text{WRONG\_ARGS})$ to misaligned {Type 2}
    end if
  end if
end for
return misaligned

B.3 Memory Drift Detection

Algorithm 3 Memory Drift Detection

Require: Trajectory τ with reasoning traces $r_1, \ldots, r_T$ and observations $o_1, \ldots, o_T$, NLI model M, confidence threshold $c = 0.85$

$\mathcal{I} \leftarrow \emptyset$ {Information state}
drift_points $\leftarrow$ []
for $t = 1$ to T **do**
  $\mathcal{I} \leftarrow \mathcal{I} \cup \{o_t\}$
  Extract factual claims $\mathcal{C}_t$ from r_t using constituency parsing
  for each claim $c_j \in \mathcal{C}_t$ **do**
    $(\text{label}, \text{conf}) \leftarrow M(c_j, \mathcal{I})$
    if label $=$ CONTRADICTION **and** conf $> c$ **then**
      Append (t, c_j, conf) to drift_points
    end if
  end for
end for
return drift_points

C IRT Model Details

C.1 Calibration Pool

The IRT model was calibrated using 8 agent configurations:

Configuration	Description
Direct × GPT-4o	Single-turn prompting
CoT × GPT-4o	Chain-of-thought
SC × GPT-4o	Self-consistency (k=5)
ReAct × GPT-4o	Interleaved reasoning/acting
Reflexion × GPT-4o	ReAct + self-reflection
RAO-X × GPT-4o	Recursive planning
ReAct × Claude-3.5-Sonnet	Cross-model calibration
ReAct × Llama-3-70B	Cross-model calibration

This yields $8 \times 578 = 4{,}624$ binary response patterns (correct/incorrect).

C.2 Model Fit

We fit the multidimensional 2-parameter logistic (M2PL) model using marginal maximum likelihood estimation via the `mirt` package [Chalmers, 2012] in R.

Table 16: IRT model fit statistics.

Statistic	Value	Acceptable Threshold
RMSEA	0.038	< 0.05
CFI	0.961	> 0.95
TLI	0.943	> 0.90

C.3 Difficulty Dimension Correlations

The moderate correlations (0.38–0.52) confirm that the three dimensions capture related but partially distinct aspects of task difficulty, justifying the multidimensional model over a unidimensional alternative.

Table 17: Pairwise correlations between IRT difficulty dimensions.

	Reasoning	Tool	Recovery
Reasoning depth	1.00	0.45	0.52
Tool complexity	0.45	1.00	0.38
Recovery demand	0.52	0.38	1.00

Table 18: Mean IRT difficulty by domain and dimension ($\pm$ std. dev.).

Domain	Reasoning	Tool	Recovery
Arithmetic	0.31 ± 0.18	0.22 ± 0.12	0.11 ± 0.08
Multi-hop QA	0.52 ± 0.21	0.48 ± 0.19	0.18 ± 0.11
API Tool Use	0.45 ± 0.19	0.58 ± 0.22	0.24 ± 0.14
Mixed Planning	0.74 ± 0.15	0.71 ± 0.17	0.38 ± 0.18

C.4 Difficulty Distribution by Domain

Mixed Planning tasks are hardest on all three dimensions, confirming the design intent. Arithmetic tasks are easiest, with particularly low recovery demand (few noise injection points). API Tool Use has the highest tool complexity, reflecting the diversity of API interaction patterns.

D Full Metric Definitions and Edge Cases

DTW alignment for Tool Alignment. We use dynamic time warping with cosine distance over tool-call embeddings. The alignment function $\pi(t)$ maps each agent step to the closest gold step, allowing for insertions and deletions. We use the FastDTW approximation [Salvador and Chan, 2007] with radius 3 for efficiency.

Recovery definition edge cases.

- If the agent fails at the last step, recovery is not possible; we exclude these from RR computation.
- If the agent encounters two failures within 3 steps, we attribute recovery (or lack thereof) to the first failure only, to avoid double-counting.
- If the agent changes strategy entirely (diverging from all gold and alternative trajectories), we count this as non-recovery even if the final answer is correct.
- If a task has zero failure points in the agent's trajectory (either no noise was injected or the agent avoided all noise points by taking a different path), that task is excluded from the agent's RR computation.

Budget Utilization for incomplete tasks. If an agent exhausts its budget without producing a final answer (TSR $= 0$), we set BU $= 0$ (worst possible). This ensures that budget exhaustion is penalized in both TSR and BU.

Tool Alignment with missing tools. If the agent calls a tool that does not appear in any gold trajectory (neither primary nor alternatives), that step receives $\text{TA}_t = 0$. If the agent skips a gold trajectory tool entirely, this is captured by the DTW alignment penalty.

E Behavioral Fingerprint Computation Details

We provide precise definitions for each of the 12 fingerprint dimensions.

1. **Exploration breadth** $= \frac{1}{N}\sum_{j=1}^{N} |\{\text{distinct tools used in } \tau_j\}|$, averaged over all tasks j and normalized by the maximum possible distinct tools for each task.
2. **Exploration depth** $= \frac{1}{N}\sum_{j=1}^{N} \frac{|\tau_j|}{b_j^{\text{reason}} \cdot B_j}$, where b_j^{reason} is the IRT reasoning difficulty and B_j is the budget. This normalizes trajectory length by both difficulty and budget.

3. **Retry tendency** $= \frac{\text{\# actions that are exact or near-duplicate of a previous action}}{\text{total \# actions}}$, where near-duplicate is defined as cosine similarity > 0.95 between action embeddings.

4. **Backtrack frequency** $= \frac{\text{\# steps where the agent explicitly undoes or reverses a previous decision}}{\text{total \# steps}}$. Detection: the agent's reasoning contains phrases like "let me try a different approach," "that was wrong," or "going back to" (matched via a curated set of 28 backtracking indicators).

5. **Tool diversity** $= \frac{H(\text{tool distribution})}{\log_2(|\text{available tools}|)}$, i.e., normalized entropy of the tool usage distribution across all tasks.

6. **Planning horizon** $= \frac{1}{N}\sum_{j=1}^{N} \frac{\text{\# consecutive same-goal steps in } \tau_j}{|\tau_j|}$, where same-goal steps are identified by consecutive steps with cosine similarity > 0.8 between their state descriptions.

7. **Error sensitivity** $= \frac{1}{|\mathcal{F}|}\sum_{t\in\mathcal{F}} \mathbb{1}[\text{strategy changes within 1 step of error at } t]$, where strategy change is defined as the next action having cosine similarity < 0.7 with the failed action.

8. **Recovery speed** $= 1 - \frac{1}{|\mathcal{F}|}\sum_{t\in\mathcal{F}} \frac{\min(\text{steps to recover from } t, 3)}{3}$. Faster recovery yields higher scores. Max 3 steps (the recovery window).

9. **Budget conservatism** $= \frac{1}{N}\sum_{j=1}^{N}\left(1 - \frac{|\tau_j|}{B_j}\right)$.

10. **Reasoning density** $= \frac{\text{total reasoning tokens across all tasks}}{\text{total reasoning tokens}+\text{total action tokens}}$.

11. **Confidence calibration** $= 1 - \text{ECE}$, where ECE is the expected calibration error computed over 10 bins of expressed confidence (extracted from reasoning traces via regex matching of confidence language: "certainly," "probably," "I think," "unsure," etc., mapped to a 5-point confidence scale).

12. **Adaptivity** $= \frac{1}{|\mathcal{D}|(|\mathcal{D}|-1)}\sum_{d_1\neq d_2} d(\mathbf{f}_i^{d_1}, \mathbf{f}_i^{d_2})$, the average pairwise fingerprint distance across domains (computed using the other 11 dimensions, excluding adaptivity itself).

F Extended Per-Domain Trajectory-Level Results

Table 19: Full trajectory-level metrics by domain: Arithmetic Reasoning (n=150).

Agent	TSR	AE	TA	RR	BU
Direct	0.59	0.88	0.61	0.05	0.82
CoT	0.71	0.82	0.68	0.10	0.76
SC	0.75	0.79	0.70	0.12	0.71
ReAct	0.76	0.65	0.78	0.28	0.52
Reflexion	0.79	0.58	0.82	0.42	0.41
RAO-X	0.83	0.74	0.85	0.58	0.64

Table 20: Full trajectory-level metrics by domain: Multi-hop QA (n=162).

Agent	TSR	AE	TA	RR	BU
Direct	0.35	0.75	0.38	0.06	0.68
CoT	0.46	0.68	0.47	0.12	0.62
SC	0.50	0.65	0.50	0.14	0.55
ReAct	0.60	0.51	0.60	0.30	0.39
Reflexion	0.64	0.44	0.65	0.44	0.28
RAO-X	0.70	0.60	0.71	0.62	0.52

Cross-domain consistency. The ranking of agents is consistent across all four domains (Spearman $\rho = 1.0$ for TSR rankings between every pair of domains). However, the *magnitude* of gaps varies substantially: the Direct→RAO-X TSR gap is 24 points on Arithmetic but 34 points on Mixed Planning, confirming that harder domains provide better discrimination.

Table 21: Full trajectory-level metrics by domain: API Tool Use (n=134).

Agent	TSR	AE	TA	RR	BU
Direct	0.36	0.72	0.35	0.09	0.66
CoT	0.46	0.66	0.44	0.15	0.59
SC	0.49	0.62	0.48	0.17	0.53
ReAct	0.57	0.49	0.58	0.32	0.37
Reflexion	0.62	0.42	0.62	0.46	0.29
RAO-X	0.66	0.58	0.68	0.63	0.50

Table 22: Full trajectory-level metrics by domain: Mixed Planning (n=132).

Agent	TSR	AE	TA	RR	BU
Direct	0.29	0.70	0.32	0.10	0.62
CoT	0.42	0.64	0.42	0.16	0.56
SC	0.46	0.60	0.46	0.19	0.49
ReAct	0.52	0.45	0.55	0.34	0.35
Reflexion	0.58	0.39	0.60	0.48	0.25
RAO-X	0.63	0.55	0.66	0.65	0.48

Recovery Rate drives domain differences. The metric with the largest cross-domain variance is RR. On Arithmetic (low noise), even Direct achieves 5% RR (few errors to recover from). On Mixed Planning (high noise), the gap between Direct (10%) and RAO-X (65%) is 55 points—the single most discriminative metric-domain combination in our evaluation.

G IRT Agent Ability Estimates

The IRT model estimates a 3-dimensional ability vector $\boldsymbol{\theta}_i \in \mathbb{R}^3$ for each agent configuration. These estimates characterize the agent's latent competence on each difficulty dimension, independent of the specific tasks in the benchmark.

Table 23: Estimated agent ability vectors from the M2PL IRT model. Higher values indicate greater competence on that dimension. Standard errors in parentheses.

Agent	θ^{reason}	θ^{tool}	θ^{recover}
Direct × GPT-4o	-0.71 (0.12)	-0.58 (0.14)	-1.24 (0.18)
CoT × GPT-4o	-0.22 (0.11)	-0.31 (0.13)	-0.82 (0.16)
SC × GPT-4o	-0.08 (0.11)	-0.18 (0.12)	-0.71 (0.15)
ReAct × GPT-4o	0.28 (0.10)	0.35 (0.11)	-0.15 (0.13)
Reflexion × GPT-4o	0.48 (0.10)	0.51 (0.11)	0.22 (0.12)
RAO-X × GPT-4o	0.82 (0.09)	0.78 (0.10)	0.91 (0.11)
ReAct × Claude-3.5	0.35 (0.10)	0.42 (0.11)	-0.08 (0.13)
ReAct × Llama-3-70B	-0.05 (0.11)	0.08 (0.12)	-0.48 (0.15)

Interpretation. Several patterns emerge:

- **Recovery is the hardest dimension for all agents.** Every agent has its lowest ability on θ^{recover}. Direct's recovery ability (-1.24) is more than a full standard deviation below its reasoning ability (-0.71), confirming that error recovery is the primary bottleneck for simple agents.
- **RAO-X's advantage is largest on recovery.** The gap between RAO-X and the next-best agent (Reflexion) is 0.34 on reasoning, 0.27 on tool use, but 0.69 on recovery. RAO-X's structured planning and reflection specifically improves error handling—the dimension where other agents are weakest.
- **Backbone LLM matters less than architecture for recovery.** ReAct × Claude-3.5 has slightly higher reasoning and tool ability than ReAct × GPT-4o ($+0.07$ each), but nearly identical recovery

(-0.08 vs. -0.15). Switching from GPT-4o to Llama-3-70B substantially reduces all abilities, with the largest drop on recovery (-0.48 vs. -0.15; $\Delta = -0.33$).

Ability profile visualization. Figure 4 shows the 3D ability profiles. The "recovery gap"—the distance between each agent's reasoning ability and its recovery ability—is a useful summary statistic: agents with smaller recovery gaps are more robust.

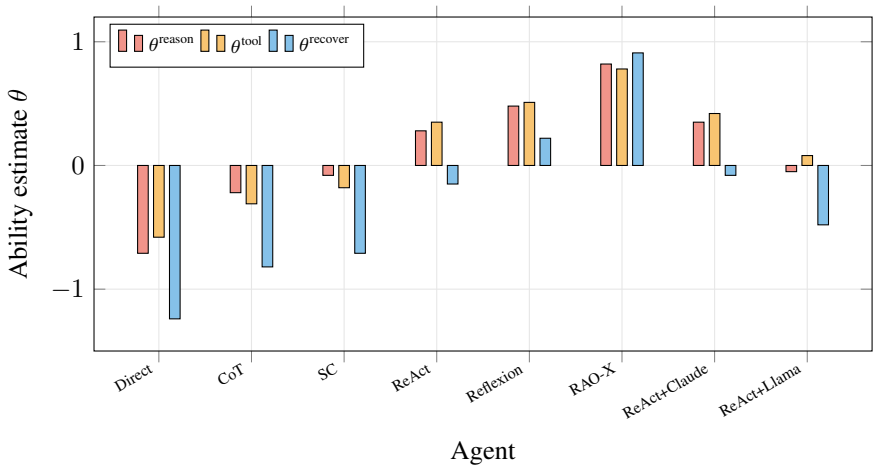

Figure 4: IRT ability estimates by agent and dimension. Recovery ability (blue) is consistently the lowest for each agent. RAO-X is the only agent where recovery exceeds reasoning, indicating a qualitative shift in capability profile.

H Failure Mode Co-occurrence Analysis

Since failure modes are not mutually exclusive, we analyze how often they co-occur within the same trajectory.

Table 24: Failure mode co-occurrence rates (proportion of failed trajectories exhibiting both modes simultaneously), aggregated across all agents.

	Oscillation	**Misalignment**	**Drift**
Oscillation	—	0.08	0.05
Misalignment	0.08	—	0.14
Drift	0.05	0.14	—

Interpretation. Co-occurrence rates are low, indicating that the three failure modes are largely independent phenomena:

- **Misalignment + Drift** is the most common co-occurrence (14%). This pattern typically manifests as: the agent drifts from observed facts, which causes it to select an inappropriate tool based on the fabricated information.
- **Oscillation + Misalignment** co-occurs in 8% of failures. This is the "wrong tool loop": the agent picks the wrong tool, gets an error, and retries the *same wrong tool* repeatedly.
- **Oscillation + Drift** is rarest (5%). These two modes are nearly orthogonal: oscillation involves repeating the same action, while drift involves changing facts. They represent opposite pathologies (stuck vs. wandering).

Agent-specific co-occurrence. Reflexion has the highest Oscillation+Misalignment co-occurrence (12%), suggesting its reflection loop sometimes fixates on retrying a misaligned tool rather than reconsidering the tool choice. RAO-X has the lowest overall co-occurrence rates (all ≤ 0.04), consistent with its failures being more diverse and less systematic (Finding 8).

I Reproducibility Checklist

- ✓ Code for task generation, agent implementations, evaluation metrics, failure detection, and fingerprinting: `https://anonymous.4open.science/r/rao-bench`
- ✓ Generated dataset (578 tasks with gold trajectories): included in repository
- ✓ Raw trajectory records for all 6 agents: included in repository
- ✓ Computed metrics, fingerprints, and IRT parameters: included as JSON files
- ✓ Human annotation guidelines and inter-annotator data: included in `annotations/` directory
- ✓ Random seeds: 42 (generation), 42+i (per-task evaluation for task i)
- ✓ Model version: `gpt-4o-2024-08-06`
- ✓ Total compute cost:
  $113.29 USD
- ✓ Evaluation runtime: approximately 14 hours (sequential, single-threaded)